

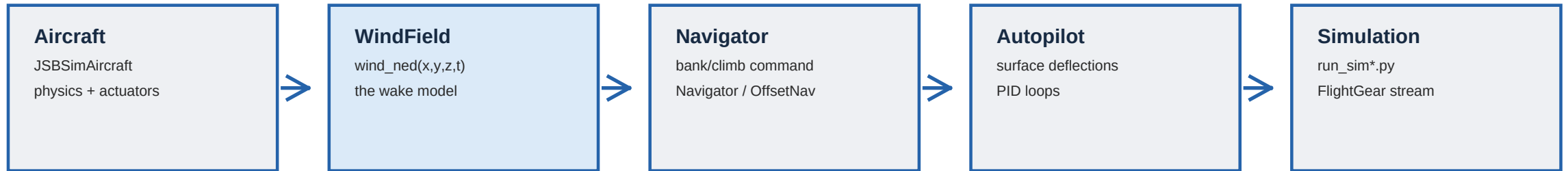
Wake Peak-Seeking

Architecture & Integration Plan

geoFS_pilot -- next step: wake-relative extremum seeking with an unseen, moving wake

1. Custom wake model 2. Sensing boundary 3. Extremum-seeking navigator 4. Wiring

Today's Architecture is Already Swappable



Each stage only depends on the previous one's output. WindField is already a clean interface -- LambOseenVortexPairWake is one implementation of it. A custom wake as a function of (x, y, t) is a new implementation of the SAME interface. Nothing else in the chain needs to change.

Plugging In a Custom Wake Model

The interface (src/atmosphere/wind_field.py)

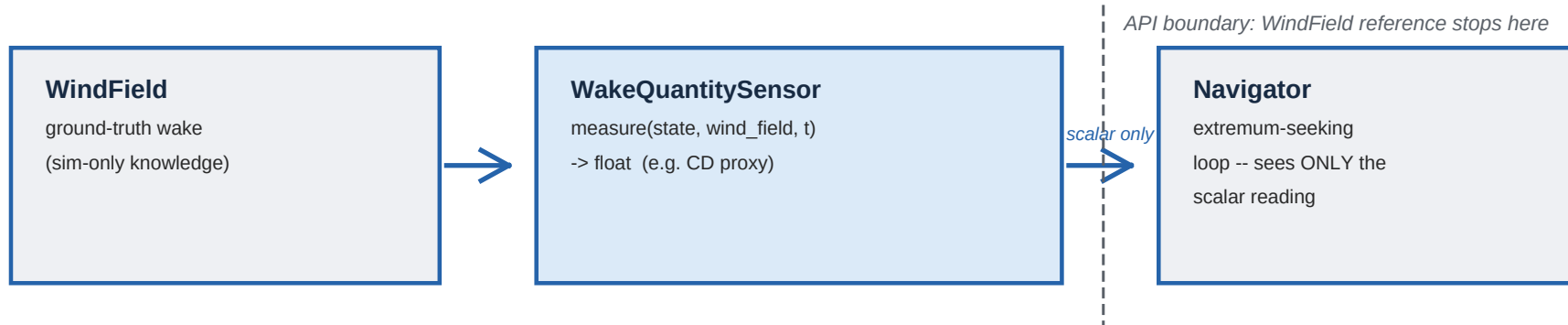
```
class WindField(ABC):  
    def wind_ned(self, x, y, z, t) -> np.ndarray:  
        # returns [Vn, Ve, Vd] m/s, NED frame
```

- x = north (m), y = east (m), z = down (m), t = time (s)
- Write a new WindField subclass anywhere -- your custom wake as a function of (x, y, t)
- Register it in src/atmosphere/factory.py's build_wind_field()
- Add a config block in config/wake.yaml

■ Aircraft, Navigator, and Autopilot need zero changes -- this boundary already exists and is exercised.

The Aircraft Can't See the Wake -- Only Feel It

A real aircraft cannot query `wind_ned()` directly. It can only feel the wake's aerodynamic effect on itself -- induced drag, lift asymmetry, required thrust. The controller must only ever see a measured scalar.



Extremum-Seeking Navigator -- a Composition, Not a Rewrite

ExtremumSeekingNavigator.compute(state, quantity)

```
dither = A * sin(omega * t)

gradient_est += lpf( quantity * sin(omega*t) )
    # demodulate the sensor reading

lateral_estimate -= k * gradient_est * dt
    # descend toward the minimum

inner_offset_nav.target_lateral_m =
    lateral_estimate + dither

return inner_offset_nav.compute(state)
```

- Wraps an internal OffsetNavigator instead of reimplementing lateral guidance
- Reuses the already-validated pure-pursuit bearing law
- As the real wake drifts, the quantity's minimum moves -- the gradient estimate re-biases and the aircraft tracks it
- Never reads `wind_field.wind_ned()` from the controller side

Wiring & Footprint

New files

src/guidance/wake_sensor.py

WakeQuantitySensor

src/guidance/extremum_seeking_navigator.py

ExtremumSeekingNavigator

One WindField subclass

your custom wake model

Touched (config/wiring only)

config/simulation.yaml

guidance_mode: "extremum_seeking"

guidance.extremum_seeking { A, omega, k, ... }

config/wake.yaml

new wake type block

run_sim*.py / dashboard.py

build sensor; pass quantity into compute()

src/sim/logger.py

+ measured_quantity_m, lateral_estimate_m